

ТЕМА: СТЕК ВЫЗОВОВ. БЫСТРАЯ СОРТИРОВКА



ВВЕДЕНИЕ

Понимание принципа работы стека вызовов и его роль в выполнении программ - невероятно важный аспект в работе с алгоритмами, так стоит знать и освоить концепцию "разделяй и властвуй" как метода решения задач, а под конец, стоит изучить алгоритм быстрой сортировки и его реализацию на практике

УРОК: СТЕК ВЫЗОВОВ

Стек вызовов

это структура данных, которая управляет вызовами функций и хранит информацию о каждом вызове, такую как параметры функции, локальные переменные и адрес возврата



Принцип работы

Push: Когда вызывается функция, информация о её вызове помещается в стек (информация о параметрах, локальных переменных, адрес возврата)

Pop: После завершения функции информация извлекается из стека, и выполнение продолжается с адреса возврата

Основные операции

Push: Добавление нового контекста вызова функции в стек

Pop: Удаление контекста функции из стека после завершения её выполнения

Peek: Просмотр текущего контекста без удаления (реализуется редко)

Пример использования стека вызовов

```
def function1():  
    print("Inside function1")  
    function2()  
  
def function2():  
    print("Inside function2")  
    function3()  
  
def function3():  
    print("Inside function3")  
  
# Вызов функции  
function1()
```

Преимущества и недостатки

Преимущества

Управление многократными вызовами функций, поддержка рекурсии

Недостатки

Ограничение по глубине стека вызовов, что может привести к ошибкам переполнения стека

Примеры проблем



Переполнение стека вызовов при глубокой рекурсии



Отладка рекурсивных функций может быть сложной из-за большого количества элементов в стеке вызовов



ВЫВОД

Мы рассмотрели основы стека вызовов. Эта концепция является важной для понимания как программные процессы управляют вызовами функций, так и для оптимизации алгоритмов сортировки. Знание этой темы поможет вам эффективно решать задачи и писать оптимизированный код

